

Audio Classification using (CNNs & Chroma-CQT) - Lab 3

Lab Overview

- Extract audio features using the Chroma-CQT representation.
- Prepare datasets for machine learning.
- Design and train a Convolutional Neural Network.
- Evaluate model performance using accuracy, loss curves, confusion matrix, and F1-score.

Contents

Lab Overview.....	1
1. Audio Feature Extraction.....	3
2. Chroma Representation	3
3. Constant-Q Transform (CQT).....	3
4. Chroma-CQT Features.....	3
5. Convolutional Neural Networks	4
6. Model Training and Evaluation.....	4
7. Evaluation Metrics.....	5
Notebook Code (Explained).....	6
Code Cell 1: Imports	6
Code Cell 2: Dataset Path	6
Code Cell 3: Chroma-CQT Settings + Feature Extraction Function.....	7
Code Cell 4: Chroma-CQT Visualization.....	8
Code Cell 5: Build Dataset Arrays (X, y)	8
Code Cell 6: Build CNN Model	9
Code Cell 7: Compile Model	10
Code Cell 8: Plot Training Curves (Accuracy & Loss).....	11
Code Cell 9: Evaluate on Test Set (Metrics + Confusion Matrix).....	12
Code Cell 10: Single Audio Inference (One File Prediction).....	13
Assignment Requirements.....	15

1. Audio Feature Extraction

Raw audio signals are sequences of amplitude values measured over time. Although humans can interpret sound easily, machine learning models require structured numerical features. Feature extraction techniques convert audio signals into representations that capture relevant properties such as pitch, rhythm, and timbre.

2. Chroma Representation

A chroma feature represents the energy distribution of the twelve pitch classes used in Western music. Each pitch class corresponds to a note name regardless of octave: C, C#, D, D#, E, F, F#, G, G#, A, A#, and B.

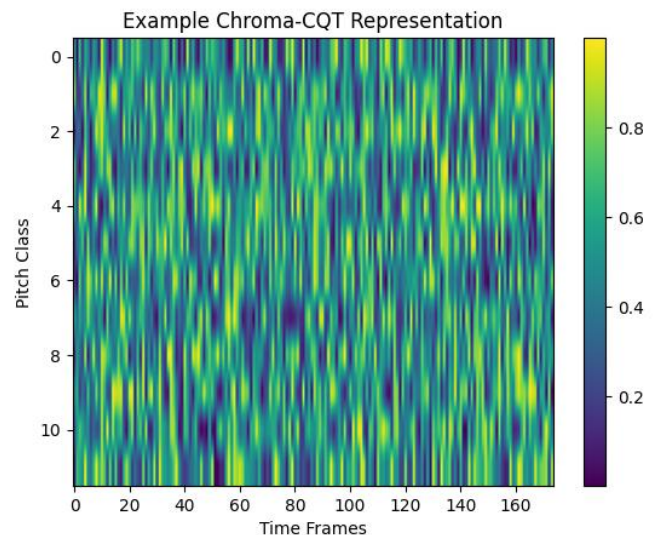
3. Constant-Q Transform (CQT)

The Constant-Q Transform is a time-frequency representation where frequency bins are spaced logarithmically rather than linearly. This matches the structure of musical scales.

where $f_{\min}$ is the minimum frequency, b is the number of bins per octave, and k is the frequency bin index.

4. Chroma-CQT Features

Chroma-CQT combines chroma representation with the Constant-Q Transform. The resulting matrix has 12 rows (one per pitch class) and multiple columns representing time frames. This representation emphasizes harmonic information, making it ideal for chord recognition.



5. Convolutional Neural Networks

Convolutional Neural Networks are deep learning models designed to learn spatial patterns in structured data. In this lab, the Chroma-CQT matrix is treated similarly to an image, where rows correspond to pitch classes and columns correspond to time.

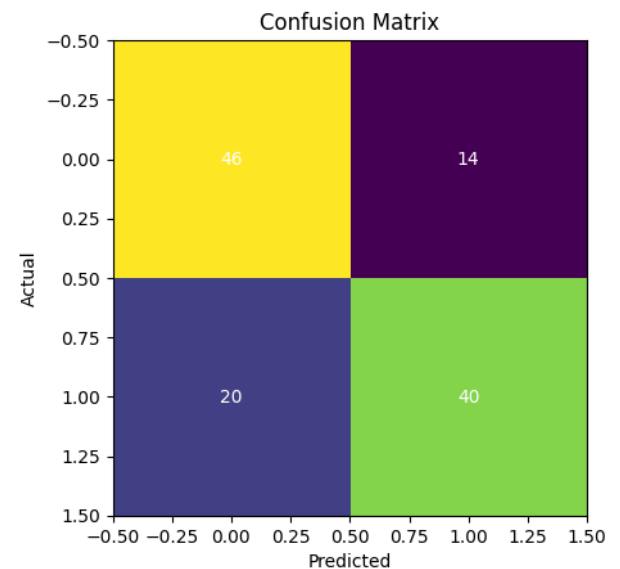
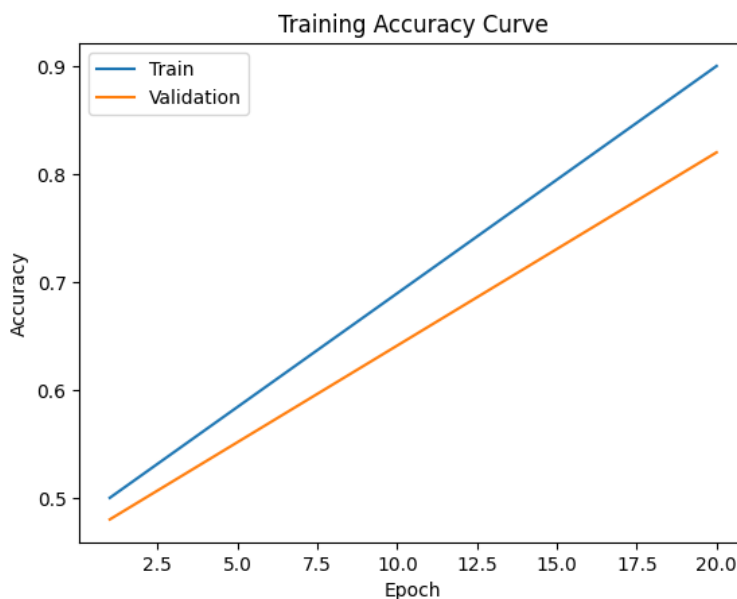
A typical CNN architecture contains the following layers:

- Convolution layers for feature extraction
- Activation functions (ReLU)
- Pooling layers for dimensionality reduction
- Fully connected layers for classification
- Softmax output layer for class probabilities



6. Model Training and Evaluation

The CNN model is trained using extracted Chroma-CQT features. During training, the model iteratively adjusts its weights to minimize categorical cross-entropy loss while maximizing classification accuracy.



7. Evaluation Metrics

Accuracy measures the proportion of correct predictions:

$$\text{Accuracy} = (\text{TP} + \text{TN}) / \text{Total}$$

Precision measures the correctness of positive predictions:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall measures how many real positives are detected:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

F1-Score combines precision and recall:

$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Notebook Code (Explained)

Notes before you run the notebook:

- Some paths in the notebook are Windows-specific (C:\Users\...). Change them to your own dataset location.

Code Cell 1: Imports

```
import os, glob, random

import numpy as np

import matplotlib.pyplot as plt

import librosa

import librosa.display

import tensorflow as tf

from tensorflow.keras import layers, models

from sklearn.model_selection import train_test_split

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report,
f1_score, accuracy_score
```

This block imports all the libraries needed for the lab in one place: Python utilities (file paths, randomization), NumPy for arrays, Matplotlib for plots, Librosa for audio loading + Chroma-CQT extraction, TensorFlow/Keras for building and training the CNN, and scikit-learn for evaluation metrics like the confusion matrix and F1-score.

Code Cell 2: Dataset Path

```
BASE_DIR = r"C:\Users\Lenovo\.cache\kagglehub\datasets\deepcontractor\musical-instrument-
chord-classification\versions\2\Audio_Files"

major_files = glob.glob(os.path.join(BASE_DIR, "Major", "*.wav"))

minor_files = glob.glob(os.path.join(BASE_DIR, "Minor", "*.wav"))

print("Major files:", len(major_files))

print("Minor files:", len(minor_files))

assert len(major_files) > 0 and len(minor_files) > 0, (
    "No WAV files found. Check BASE_DIR. It should contain folders: Major/ and Minor/"
```

```
)  
  
random.seed(42)  
  
random.shuffle(major_files)  
  
random.shuffle(minor_files)
```

This block defines the dataset folder location (where the audio files are stored) and collects the file paths for each class (Major and Minor). It also typically verifies that the folders exist and that files are found, so you don't train on an empty dataset by mistake.

Code Cell 3: Chroma-CQT Settings + Feature Extraction Function

```
SR = 22050  
  
HOP_LENGTH = 512  
  
N_CHROMA = 12  
  
MAX_FRAMES = 174  
  
def extract_chroma_cqt(path, sr=SR, hop_length=HOP_LENGTH, n_chroma=N_CHROMA,  
max_frames=MAX_FRAMES):  
  
    y, sr = librosa.load(path, sr=sr)  
  
    chroma = librosa.feature.chroma_cqt(y=y, sr=sr, hop_length=hop_length,  
n_chroma=n_chroma) # (n_chroma, time)  
  
    if chroma.shape[1] < max_frames:  
        chroma = np.pad(chroma, ((0, 0), (0, max_frames - chroma.shape[1])), mode="constant")  
  
    else:  
        chroma = chroma[:, :max_frames]  
  
    chroma = (chroma - chroma.mean()) / (chroma.std() + 1e-9)  
  
    return chroma.astype(np.float32)  
  
test_path = major_files[0]  
  
chroma_feat = extract_chroma_cqt(test_path)  
  
print("Test file:", test_path)
```

```
print("Chroma-CQT shape:", chroma_feat.shape)
```

This block defines the parameters used for Chroma-CQT (sample rate, hop length, number of chroma bins, and fixed number of time frames) and implements the feature-extraction function. The function loads an audio file, computes the Chroma-CQT matrix, then pads/trims it to a consistent shape so every sample can be fed into the CNN.

Code Cell 4: Chroma-CQT Visualization

```
plt.figure(figsize=(10, 4))

librosa.display.specshow(chroma_feat, x_axis="time", y_axis="chroma", sr=SR,
hop_length=HOP_LENGTH)

plt.colorbar()

plt.title("Chroma-CQT (normalized)")

plt.tight_layout()

plt.show()
```

This block takes one audio sample, extracts its Chroma-CQT features, and plots them as a heatmap. The goal is to visually confirm that the representation looks correct (12 pitch-class rows across time) before moving on to dataset building and training.

Code Cell 5: Build Dataset Arrays (X, y)

```
N_PER_CLASS = 300 # change to None to use all available

maj = major_files if N_PER_CLASS is None else major_files[:min(N_PER_CLASS,
len(major_files))]

minr = minor_files if N_PER_CLASS is None else minor_files[:min(N_PER_CLASS,
len(minor_files))]

files = maj + minr

labels = [0] * len(maj) + [1] * len(minr) # 0=Major, 1=Minor
```



```
print("Using files:", len(files), "(Major:", len(maj), "Minor:", len(minr), ")")

# Extract features (can take a bit depending on N_PER_CLASS)

X = np.array([extract_chroma_cqt(f) for f in files], dtype=np.float32)

X = X[..., np.newaxis] # add channel dim -> (N, 12, 174, 1)

y = tf.keras.utils.to_categorical(labels, num_classes=2)

print("X shape:", X.shape)

print("y shape:", y.shape)

X_train, X_test, y_train, y_test, labels_train, labels_test = train_test_split(
    X, y, labels, test_size=0.2, random_state=42, stratify=labels
)
```

This block chooses how many samples per class to use, shuffles the file lists, extracts Chroma-CQT features for all files, and constructs the final dataset arrays. It outputs X as the input feature tensors and y as the labels (typically one-hot encoded), then reshapes X to include a channel dimension so it matches CNN input requirements.

Code Cell 6: Build CNN Model

```
tf.keras.backend.clear_session()

sound_model = models.Sequential([

    layers.Input(shape=(N_CHROMA, MAX_FRAMES, 1)),

    layers.Conv2D(16, (3, 3), activation="relu", padding="same"),

    layers.MaxPool2D((2, 2)),

    layers.Conv2D(32, (3, 3), activation="relu", padding="same"),

    layers.MaxPool2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation="relu", padding="same"),

    layers.MaxPool2D((2, 2)),
```

```
layers.GlobalAveragePooling2D(),  
layers.Dense(64, activation="relu"),  
layers.Dropout(0.3),  
layers.Dense(2, activation="softmax")  
])  
sound_model.compile(  
    optimizer="adam",  
    loss="categorical_crossentropy",  
    metrics=["accuracy"]  
)  
sound_model.summary()
```

This block clears any previous TensorFlow state, splits the dataset into training and testing sets, and builds the CNN architecture. It defines the input shape, convolution/pooling layers to learn patterns from the Chroma-CQT “image,” then ends with dense layers and a softmax output for Major vs Minor classification.

Code Cell 7: Compile Model

```
EPOCHS = 20  
BATCH_SIZE = 32  
history = sound_model.fit(  
    X_train, y_train,  
    validation_split=0.2,  
    epochs=EPOCHS,  
    batch_size=BATCH_SIZE,  
    verbose=1  
)
```

This block compiles the CNN by setting the optimizer (usually Adam), the loss function (cross-entropy for classification), and metrics (accuracy). Then it trains the model for a chosen number of epochs and stores the training history so you can plot learning curves afterward.

Code Cell 8: Plot Training Curves (Accuracy & Loss)

```
plt.figure()

plt.plot(history.history["accuracy"])

plt.plot(history.history["val_accuracy"])

plt.title("Accuracy")

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.legend(["train", "val"])

plt.show()

plt.figure()

plt.plot(history.history["loss"])

plt.plot(history.history["val_loss"])

plt.title("Loss")

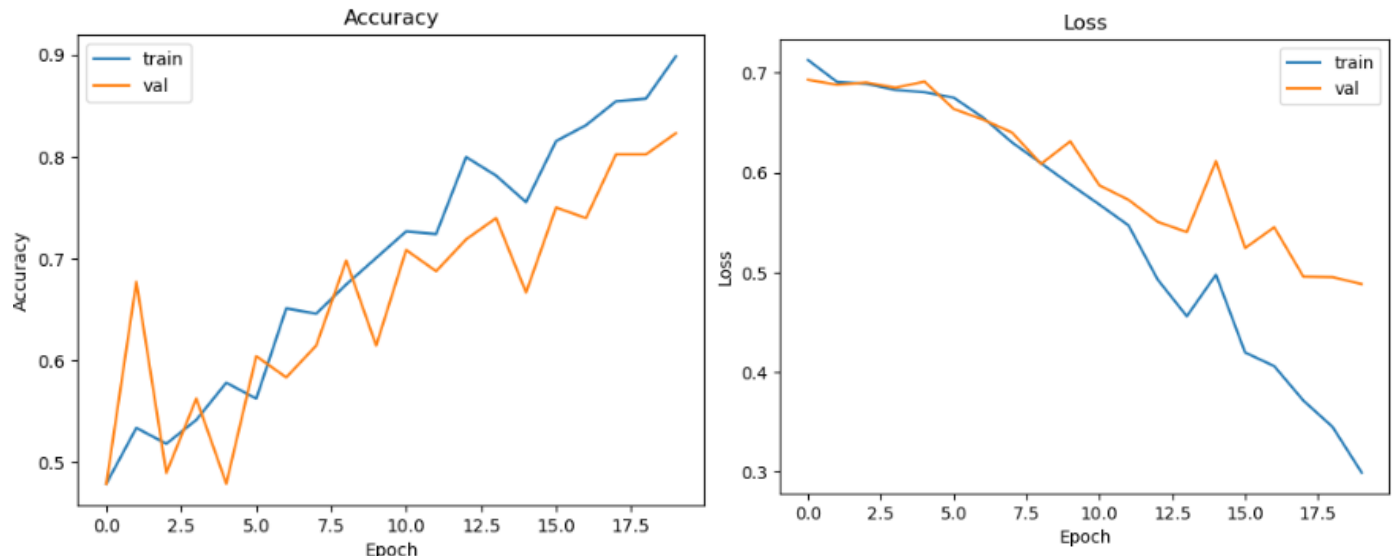
plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend(["train", "val"])

plt.show()
```

This block uses the saved training history to plot training vs validation accuracy and loss across epochs. These plots help you judge whether the model is learning properly and whether overfitting is happening



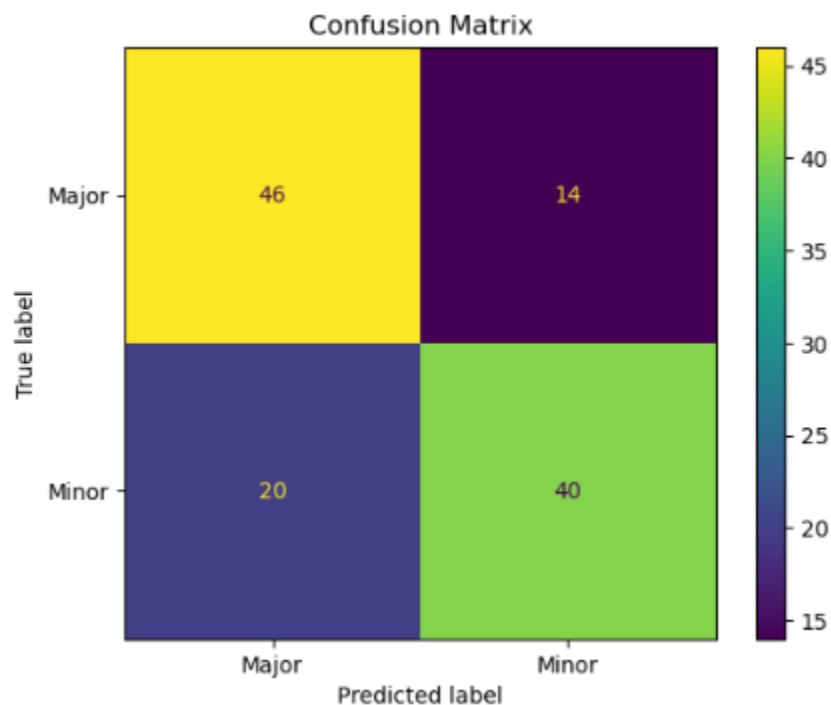
(training improves while validation stalls or worsens).

Code Cell 9: Evaluate on Test Set (Metrics + Confusion Matrix)

```
y_prob = sound_model.predict(X_test, verbose=0)
y_pred = np.argmax(y_prob, axis=1)
y_true = np.argmax(y_test, axis=1)
acc = accuracy_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred, average="macro")
print("Test Accuracy:", acc)
print("Macro F1-score:", f1)
cm = confusion_matrix(y_true, y_pred, labels=[0, 1])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Major", "Minor"])
disp.plot()
plt.title("Confusion Matrix")
plt.show()
print("\nClassification Report:")
```

```
print(classification_report(y_true, y_pred, target_names=["Major", "Minor"]))
```

This block runs the trained model on the test data, converts predicted probabilities into class labels, and computes evaluation metrics such as test accuracy, macro F1-score, a classification report, and the confusion matrix. It then visualizes the confusion matrix to show exactly how many Major/Minor samples were correctly or incorrectly classified.



Code Cell 10: Single Audio Inference (One File Prediction)

```
sample_path = minor_files[0]

sample_feat = extract_chroma_cqt(sample_path)[np.newaxis, ..., np.newaxis] # (1, 12, 174, 1)

probs = sound_model.predict(sample_feat, verbose=0)[0]

pred_idx = int(np.argmax(probs))

pred_label = "Major" if pred_idx == 0 else "Minor"

print("File:", sample_path)
```

```
print("Probabilities [Major, Minor]:", probs)

print("Prediction:", pred_label)
```

This final block picks a single audio file, extracts its Chroma-CQT features in the same way as training, and passes it to the CNN to predict whether it is Major or Minor. It prints the predicted label (and sometimes probabilities), which demonstrates how the trained model can be used on new unseen audio inputs.

```
Probabilities [Major, Minor]: [0.01143128 0.9885688]
Prediction: Minor
```

Assignment Requirements

Students must submit a report demonstrating:

- Correct dataset preparation
- Chroma-CQT feature extraction
- CNN model implementation
- Training and evaluation results
- Interpretation of accuracy, loss curves, and confusion matrix